3D IT Park Visualization

CSE 4208: Computer Graphics Laboratory

Submitted By:

Dipto Saha
Roll: 2007119
Year: 4th
Semester: 2nd

Submitted To:

Dr. Sk. Md. Masudul Ahsan
Professor

Md Tajmilur Rahman
Lecturer

Department of Computer Science and Engineering
Khulna University of Engineering & Technology, Khulna

April, 2026

Contents

1 Project Title **3**

2 Abstract **3**

3 Introduction **3**

4 Objectives **4**

5 Theoretical Background **4**

5.1 OpenGL Rendering Pipeline 4

5.2 Model Transformation 5

5.3 View Transformation and Custom Camera 5

5.4 Projection Transformation 6

5.5 Lighting Model 6

5.6 Texture Mapping 7

5.7 Bezier Curve Based Modeling 7

5.8 Fractal Geometry 7

5.9 Environmental Motion 7

6 System Design and Implementation **8**

6.1 Software Environment 8

6.2 Scene Composition 8

6.3 Primitive and Hierarchical Modeling 10

6.4 Lighting and Shader Implementation 11

6.5 Texture Mapping Implementation 12

6.6 Bezier Vase Generation 12

6.7 Fractal Tree Generation 13

6.8 Animation and Environmental Motion 14

6.9 Camera Interaction 14

7 Results and Discussion **15**

8 Conclusion **15**

9 References **15**

1 Project Title

The title of this project is *3D IT Park Visualization.* The work presents a complete real-time architectural environment developed using Modern OpenGL. The scene includes both interior and exterior regions of a modern IT park and demonstrates important concepts of computer graphics through modeling, transformation, shading, texture mapping, curve-based surface generation, recursive structure generation, and interactive visualization.

2 Abstract

This report presents the design and implementation of a real-time three-dimensional IT park visualization developed using C++, OpenGL, GLFW, GLAD, GLM, and GLSL shaders. The environment contains classrooms, a server room, a library room, a reception room, outdoor landscape, a swimming pool area, decorative curved objects, and procedural trees. The project emphasizes hierarchical modeling, shader-based illumination, texture mapping, custom camera control, procedural geometry generation through a cubic Bezier curve, recursive fractal construction, and environmental rain animation. The rendering system supports both Phong and Gouraud shading, multiple light sources, texture blending modes, and interactive navigation. The completed system demonstrates how mathematical formulations and graphics programming techniques can be integrated to create a visually rich and functionally interactive architectural scene.

3 Introduction

Architectural visualization in computer graphics allows users to understand spatial structure, material appearance, illumination, and interaction in a way that static diagrams cannot provide. This project was developed to create a modern IT park as a real-time navigable environment rather than as a fixed rendering. The scene includes multiple functional spaces such as classrooms, a server room, a library room, and a reception room, together with outdoor features such as a pool area, vegetation, decorative objects, and surrounding pathways. The user can move through the scene and observe lighting variations, texture modes, shading differences, animation, and weather effects.

The project is implemented using the programmable OpenGL pipeline, where vertex and fragment shaders control transformation and illumination. In addition to manually assembled geometry, the project includes mathematically generated content such as a Bezier-curve based vase, recursive fractal trees, and time-based rain motion. These components make the project a broader demonstration of practical graphics concepts rather than a simple static scene.

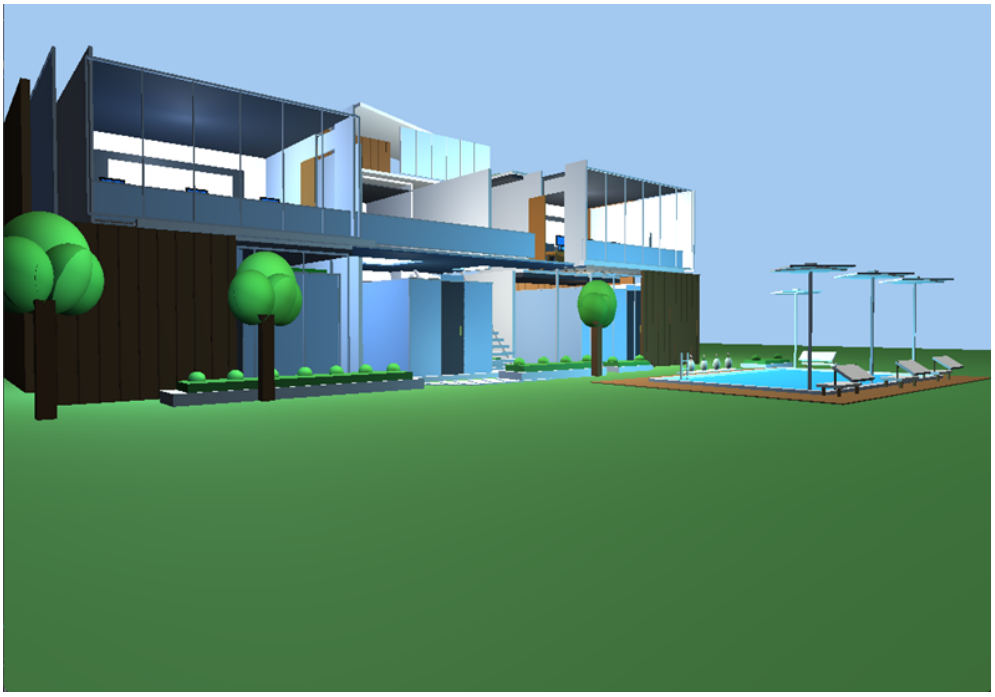

Figure 1: Overall outer environment of the 3D IT Park.

4 Objectives

- To build a complete 3D visualization of a modern IT park.
- To combine indoor and outdoor scene modeling in one environment.
- To apply transformation, lighting, shading, and texture mapping techniques.
- To implement procedural and recursive geometry for added realism.
- To support interactive camera navigation through the scene.
- To compare Phong and Gouraud shading methods.
- To demonstrate different lighting components and texture modes at runtime.
- To use mathematical and time-based techniques for realistic visual effects.

5 Theoretical Background

5.1 OpenGL Rendering Pipeline

The project follows the programmable OpenGL rendering pipeline. A local vertex is transformed to world space by the model matrix, then to camera space by the view matrix, and finally to

clip space by the projection matrix. The combined transformation is written as

$$\mathbf{p}_{clip} = \mathbf{M}_{proj}\mathbf{M}_{view}\mathbf{M}_{model}\mathbf{p}_{local}. \tag{1}$$

After this stage, clipping, rasterization, interpolation, and fragment processing are applied. In this project, the vertex shader computes world-space position, transformed normal, and texture coordinate, while the fragment shader computes the final illuminated color of each fragment.

5.2 Model Transformation

Each scene object is positioned by combining translation, rotation, and scaling. If $\mathbf{T}$ represents translation, $\mathbf{R}_x$, $\mathbf{R}_y$, and $\mathbf{R}_z$ represent rotation matrices, and $\mathbf{S}$ represents scaling, then the model matrix is

$$\mathbf{M}_{model} = \mathbf{T}\mathbf{R}_x\mathbf{R}_y\mathbf{R}_z\mathbf{S}. \tag{2}$$

This same ordering is used repeatedly in the project to assemble complex objects from primitive components.

5.3 View Transformation and Custom Camera

The project uses a custom camera system. The view matrix may be obtained directly by

$$\mathbf{M}_{view} = \texttt{lookAt}(\mathbf{eye}, \mathbf{eye} + \mathbf{Front}, \mathbf{Up}), \tag{3}$$

and the project also contains a manual construction of the same transformation. If $\mathbf{E}$ is the eye position, $\mathbf{C}$ is the look target, and $\mathbf{U}$ is the world up vector, then the camera basis vectors are

$$\mathbf{z} = \frac{\mathbf{E} - \mathbf{C}}{\|\mathbf{E} - \mathbf{C}\|}, \qquad \mathbf{x} = \frac{\mathbf{U} \times \mathbf{z}}{\|\mathbf{U} \times \mathbf{z}\|}, \qquad \mathbf{y} = \mathbf{z} \times \mathbf{x}. \tag{4}$$

The custom view transformation is therefore expressed as

$$\mathbf{M}_{view} = \mathbf{R}\mathbf{T}, \tag{5}$$

where $\mathbf{R}$ is the rotation matrix formed from $\mathbf{x}, \mathbf{y}, \mathbf{z}$ and $\mathbf{T}$ is the translation matrix of the eye position.

The front vector of the camera is updated from yaw and pitch angles using

$$F_x = \cos(\psi)\cos(\theta), \qquad F_y = \sin(\theta), \qquad F_z = \sin(\psi)\cos(\theta), \tag{6}$$

where ψ is yaw and θ is pitch. This allows the user to control the camera direction interactively.

5.4 Projection Transformation

The project uses perspective projection. If the field of view is f, the aspect ratio is a, the near plane is n, and the far plane is f_r, then the perspective projection matrix is

$$\mathbf{M}_{proj} = \begin{bmatrix} \frac{1}{a\tan(f/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(f/2)} & 0 & 0 \\ 0 & 0 & \frac{f_r+n}{n-f_r} & \frac{2f_r n}{n-f_r} \\ 0 & 0 & -1 & 0 \end{bmatrix}. \tag{7}$$

This matrix is used in the project through the camera projection helper with configurable near and far plane values.

5.5 Lighting Model

The illumination model used in the project is based on ambient, diffuse, and specular reflection, together with an emissive term for glowing objects. The total intensity at a surface point is

$$I = I_e + I_a + I_d + I_s. \tag{8}$$

The ambient term is

$$I_a = \mathbf{k}_a \odot \mathbf{L}_a, \tag{9}$$

the diffuse term is

$$I_d = \mathbf{k}_d \odot \mathbf{L}_d \max(0, \mathbf{n} \cdot \mathbf{l}), \tag{10}$$

and the specular term is

$$I_s = \mathbf{k}_s \odot \mathbf{L}_s \left[\max(0, \mathbf{r} \cdot \mathbf{v})\right]^{\alpha}, \tag{11}$$

where $\mathbf{n}$ is the unit normal, $\mathbf{l}$ is the light direction, $\mathbf{v}$ is the view direction, $\mathbf{r}$ is the reflection direction, and α is the shininess factor.

For point lights and spotlights, attenuation is used:

$$A(d) = \frac{1}{k_c + k_l d + k_q d^2}, \tag{12}$$

where d is the distance from the light to the fragment.

The project supports both Phong and Gouraud shading. In Phong shading, lighting is evaluated per fragment. In Gouraud shading, ambient, diffuse, and specular components are first computed in the vertex shader and then interpolated across the surface.

5.6 Texture Mapping

Texture mapping is used for selected surfaces such as doors, grass, and walls. The project supports three modes: material-only mode, texture-only mode, and blended mode. If the sampled texture color is $\mathbf{T}(u, v)$ and the material diffuse color is $\mathbf{k}_d$, then the effective diffuse color becomes

$$\mathbf{k}_d^{eff} = \begin{cases} \mathbf{k}_d, & \text{material only,} \\ \mathbf{T}(u, v), & \text{texture only,} \\ \mathbf{k}_d \odot \mathbf{T}(u, v), & \text{blended mode.} \end{cases} \tag{13}$$

5.7 Bezier Curve Based Modeling

A decorative vase is created using a cubic Bezier curve. If the four control points are $\mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2$, and $\mathbf{P}_3$, then the cubic Bezier curve is

$$\mathbf{B}(t) = (1-t)^3\mathbf{P}_0 + 3(1-t)^2 t\mathbf{P}_1 + 3(1-t)t^2\mathbf{P}_2 + t^3\mathbf{P}_3, \qquad 0 \le t \le 1. \tag{14}$$

Its tangent is

$$\mathbf{B}'(t) = 3(1-t)^2(\mathbf{P}_1 - \mathbf{P}_0) + 6(1-t)t(\mathbf{P}_2 - \mathbf{P}_1) + 3t^2(\mathbf{P}_3 - \mathbf{P}_2). \tag{15}$$

The sampled curve is revolved around the y-axis to generate the final three-dimensional surface:

$$\mathbf{p}(t, \phi) = \begin{bmatrix} r(t)\cos\phi \\ y(t) \\ -r(t)\sin\phi \end{bmatrix}. \tag{16}$$

5.8 Fractal Geometry

The outdoor tree is generated recursively. Each branch produces smaller branches until a fixed recursion depth is reached. In this project, the fractal depth is four. The recursive branching makes the tree visually complex while keeping the logic compact and mathematically structured.

5.9 Environmental Motion

Rain is implemented as a repeated environmental motion effect rather than as a single moving object. Many droplets are distributed across the exterior region, and their positions are slightly varied by a stable noise function to avoid a rigid pattern. If the droplet height is updated over time by the scene clock, the motion can be expressed conceptually as

$$y(t) = H - \mathrm{mod}(vt + \phi, H), \tag{17}$$

where H is the fall height, v is the falling speed, and ϕ is a phase offset. This produces the visual effect of continuous falling rain.

6 System Design and Implementation

6.1 Software Environment

The project is implemented in C++ using OpenGL 3.3 core profile. GLFW is used for window creation and input management, GLAD is used to load OpenGL functions, GLM is used for vector and matrix operations, and STB Image is used for loading textures. The programmable rendering pipeline is implemented with GLSL shaders. Depth testing, back-face culling, blending, and vertical synchronization are enabled to improve rendering correctness and visual quality.

6.2 Scene Composition

The scene is divided into both outdoor and indoor components. The outdoor region includes the surrounding ground, paved pathways, decorative planters, vegetation, a pool area, lounge furniture, and umbrellas. The indoor region includes a server room, a library room, a reception room, central atrium space, upper classrooms, and support spaces. This decomposition allows the project to remain logically organized while still representing a large and detailed environment.

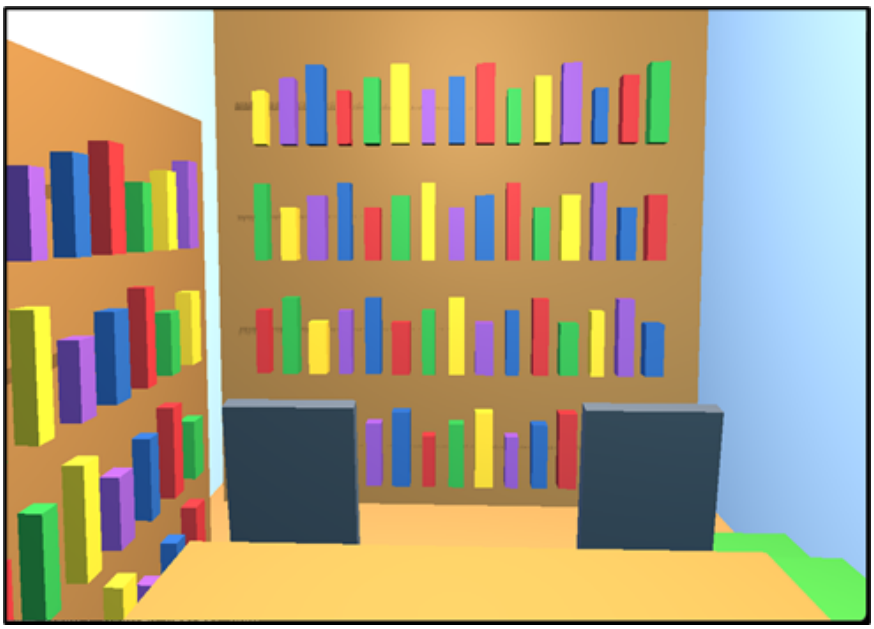

Figure 2: Library room of the IT Park.

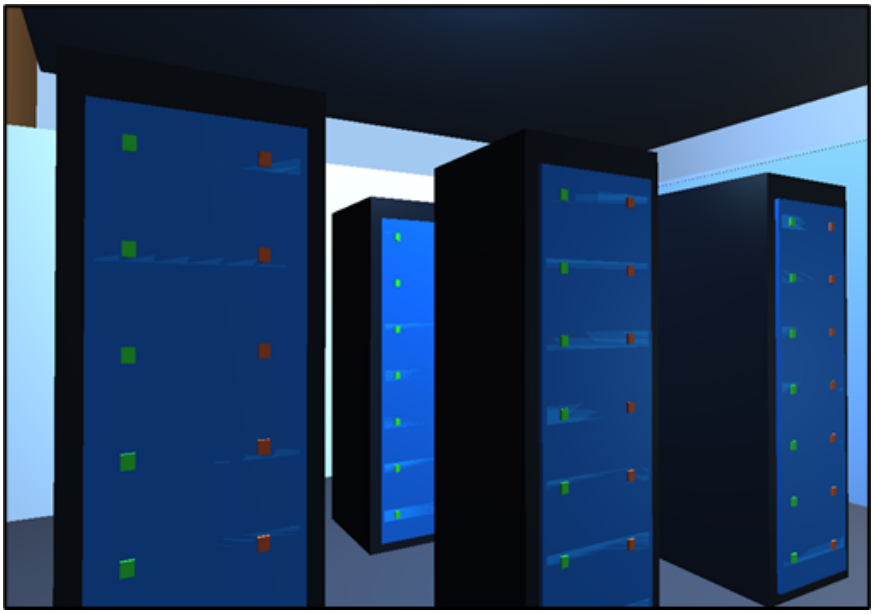

Figure 3: Server room of the IT Park.

Figure 4: Upper floor classroom.

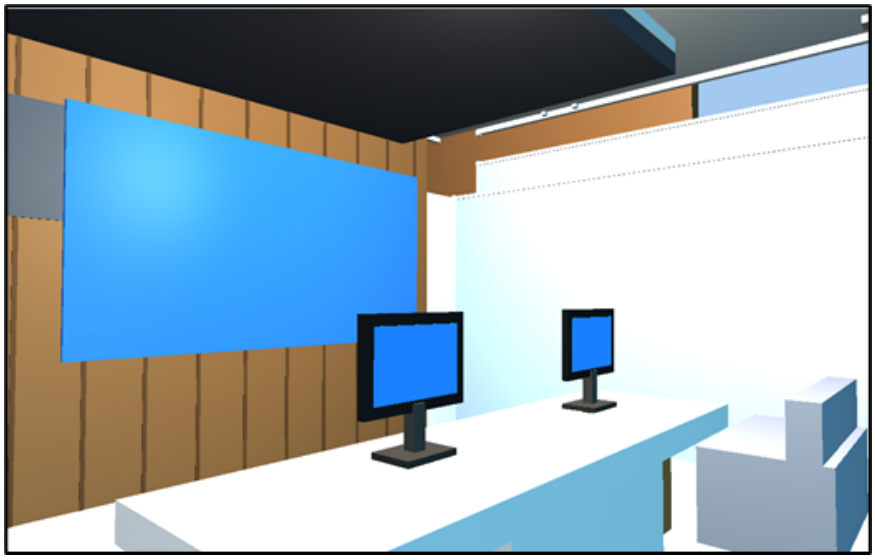

Figure 5: Reception room of the IT Park.

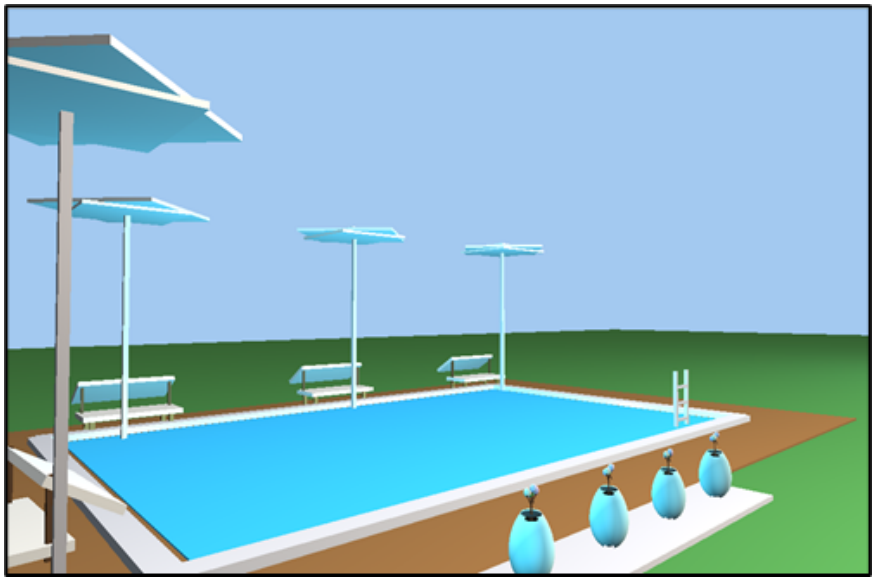

Figure 6: Swimming pool and surrounding outdoor space.

6.3 Primitive and Hierarchical Modeling

Most objects in the project are formed by composing primitive cubes and spheres through hierarchical transformations. Furniture such as desks, monitors, keyboards, chairs, reception counters, and poolside objects are assembled from multiple cuboid and spherical parts. A reusable indexed cube mesh is stored once and drawn repeatedly with different model transformations. This reduces redundancy and makes the design modular.

6.4 Lighting and Shader Implementation

The lighting system is one of the major technical features of the project. A directional light simulates broad scene illumination, seven point lights are placed at important functional locations such as the reception room, classrooms, server room, library-atrium region, and poolside area, and a spotlight follows the camera direction. The shaders allow independent enabling or disabling of ambient, diffuse, specular, and emissive contributions, making it possible to analyze the influence of each component separately.

The project also supports both Phong and Gouraud shading. In Gouraud mode, lighting values are computed in the vertex shader and interpolated across the surface. In Phong mode, normals and fragment positions are interpolated first and lighting is computed in the fragment shader, producing more accurate highlights.

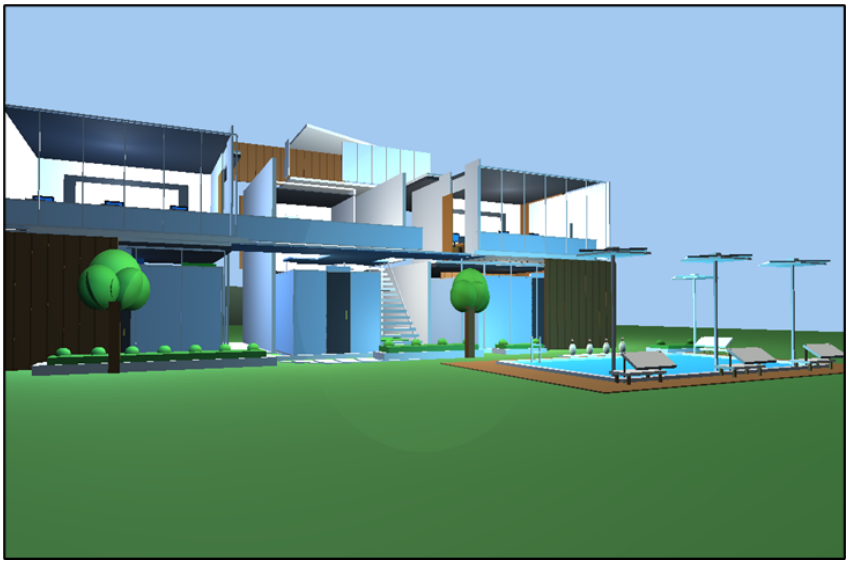

Figure 7: Rendering with Phong shading.

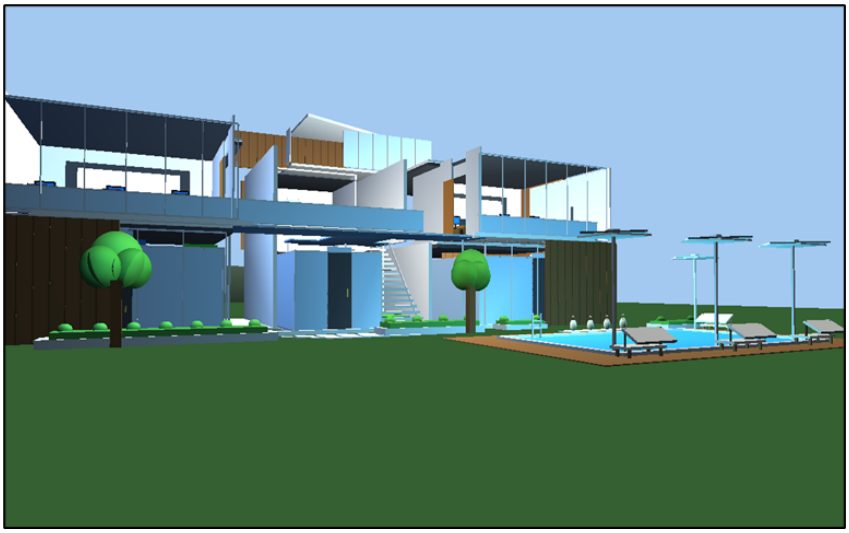

Figure 8: Rendering with Gouraud shading.

6.5 Texture Mapping Implementation

Texture mapping is applied selectively to provide stronger surface detail and realism. Door panels use a door texture, the outdoor ground uses a grass texture, and selected wall surfaces use a wall texture. The fragment shader supports material-only mode, texture-only mode, and blended mode. This makes the application useful not only as a final scene but also as an interactive demonstration of how texture affects material appearance.

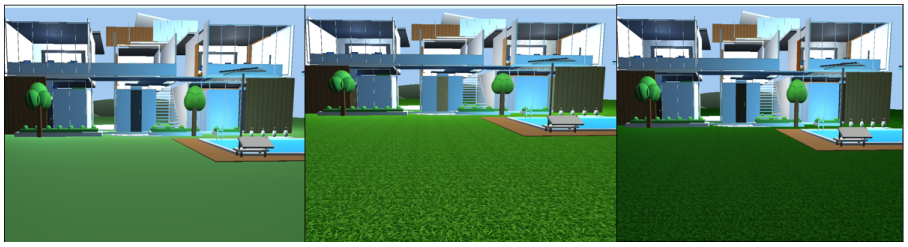

Figure 9: Comparison of material-only, texture-only, and blended texture modes.

6.6 Bezier Vase Generation

The Bezier vase is generated procedurally rather than being manually modeled. Four control points define the two-dimensional side profile. The curve is sampled at multiple parameter values, and the tangent at each sample is used to estimate a normal. The profile is then rotated around the vertical axis through several slices, creating a full three-dimensional mesh. The mesh data is stored through VAO, VBO, and EBO structures. After generating the vase body, extra decorative components such as the lip, base, soil, and flowers are attached using primitive geometry.

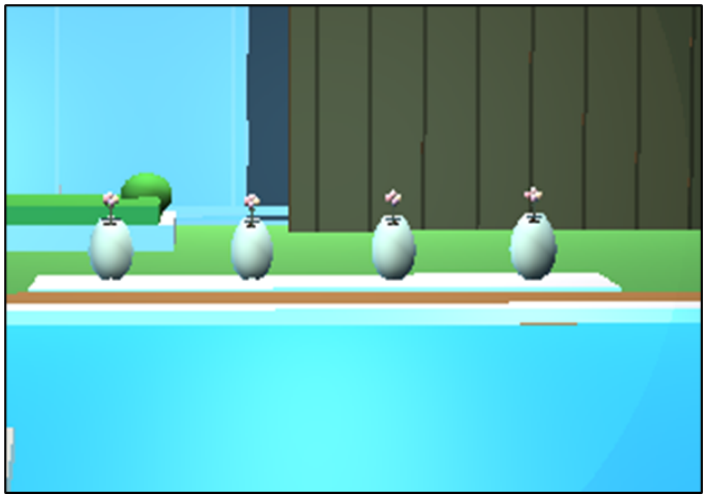

Figure 10: Bezier-curve based vase used in the project.

6.7 Fractal Tree Generation

The fractal tree is created using a recursive branching function. A branch is drawn first, and then smaller branches are attached with different rotations. This recursive process continues until the terminal depth is reached, where leaf spheres are added. The result is a visually rich object generated from a short mathematical pattern.

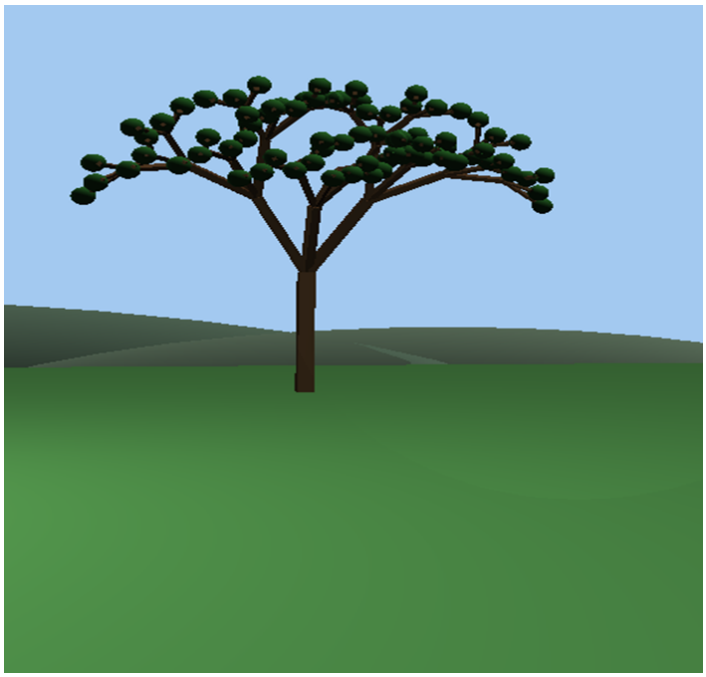

Figure 11: Fractal tree generated by recursive branching.

6.8 Animation and Environmental Motion

The project contains several animated components. Doors in multiple rooms can open and close, and a roof window also changes angle during interaction. If θ_c is the current angle, θ_t is the target angle, s is the animation speed, and Δt is the frame time, then the update rule may be written as

$$\theta_{new} = \theta_c + (\theta_t - \theta_c)s\Delta t. \tag{18}$$

This produces a smooth interpolation from one state to another.

Rain is implemented as a distributed set of droplets over the outdoor area. Each droplet is rendered as a small elongated cuboid, and the vertical position of the droplets is updated continuously over time. The rain effect increases the realism of the scene and demonstrates time-dependent environmental motion in the application.

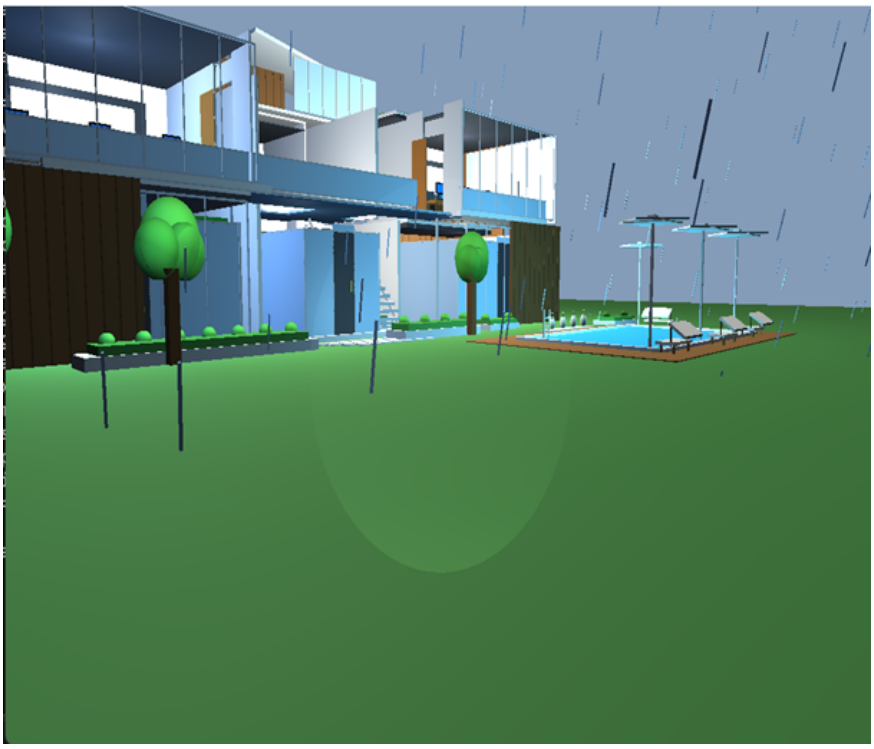

Figure 12: Rain effect rendered as environmental motion.

6.9 Camera Interaction

The project uses a custom camera class that supports forward, backward, left, right, upward, and downward motion. The camera direction is controlled by mouse movement as well as by keyboard-based yaw and pitch adjustment. Zoom is also supported through mouse scroll. In addition to the standard `lookAt` formulation, the project includes a manual view matrix construction, which reinforces the mathematical interpretation of camera transformation. Another useful feature is floor-aware navigation, where the camera height is constrained relative to the

current walkable surface so that movement through the IT park remains realistic.

7 Results and Discussion

The completed project demonstrates that a large architectural environment can be built effectively by combining reusable primitives, mathematically generated geometry, shader-based illumination, and interactive control. The indoor rooms and outdoor spaces remain visually distinguishable, and the real-time camera allows the user to inspect the design from multiple viewpoints. The lighting system enables meaningful comparison of Phong and Gouraud shading as well as ambient, diffuse, and specular contributions. Texture support improves material variety, while the Bezier vase and fractal tree show how mathematical methods can enrich a modeled scene. Rain and animated structural elements provide additional dynamism.

A key challenge in the project was maintaining smooth navigation and rendering quality in a large multi-room environment. This challenge was addressed by organizing the scene into separate functions, using a structured camera system, and allowing lighting and shading modes to be switched interactively for analysis. The resulting scene is both demonstrative and visually coherent.

8 Conclusion

This project successfully presents a complete three-dimensional visualization of a modern IT park using Modern OpenGL. It integrates scene modeling, hierarchical transformation, lighting, shading, texture mapping, procedural geometry generation, recursion, animation, and user interaction in a single real-time environment. The project also demonstrates how mathematical equations related to transformation, illumination, parametric curves, and time-based motion can be translated directly into practical graphics programming. The final output is a detailed and interactive virtual scene that serves as a strong demonstration of core computer graphics concepts.

9 References

References

[1] Joey de Vries, *Learn OpenGL*, https://learnopengl.com/

[2] Dave Shreiner, Graham Sellers, John Kessenich, and Bill Licea-Kane, *OpenGL Programming Guide*, Addison-Wesley

[3] GLM Documentation, https://github.com/g-truc/glm

[4] GLFW Documentation, https://www.glfw.org/documentation.html

[5] Edward Angel and Dave Shreiner, *Interactive Computer Graphics: A Top-Down Approach with OpenGL*, Addison-Wesley

[6] *Lab Slide*